Sri Lanka Institute of Information Technology

BSc. (Hons) in Information Technology
Specialized in Cyber Security
Year 3, Semester 2

IE3092– Information Security Project

Clipboard Security Manager

Submitted By

Student ID No.	Name
IT23136106	D.M.M. Pasindu Supushmika
IT23271500	J.T. Mithum

CLIPBOARD SECURITY MANAGER: A CROSS-PLATFORM SYSTEM FOR SENSITIVE DATA ENCRYPTION AND MASKING

Bachelor of Science (Hons) in Information Technology Specialized in Cyber Security

T.M. Jayalingam

Bachelor of Science (Hons) in Information Technology Specialized in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

February 2026

DECLARATION OF THE CANDIDATE

We declare that this is our own work and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
D. M. M. Pasindu Supushmika Mahamohottala	**IT23136106**	
J. T. Jayalingam	**IT2327500**	

The supervisor/s should certify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

_____ _____

Signature of the supervisor Date

ABSTRACT

The system clipboard is a critical yet often overlooked vulnerability in modern computing environments, as it typically stores sensitive data in unencrypted plain text. This project introduces the "Clipboard Security Manager," a lightweight, cross-platform background application designed to mitigate the risks of both intentional data leaks and unintentional clipboard hijacking. The methodology involves using the Go (Golang) programming language to implement a system-level hook that intercepts copy commands and immediately encrypts data using the AES-256-GCM algorithm. To prevent unauthorized pasting, the software utilizes a mandatory confirmation pop-up and a secure password verification process backed by Argon2 hashing. Preliminary testing focuses on the accuracy of the encryption-decryption cycle and the reliability of the tiered security response, which includes email notifications and temporary authentication lockouts after three failed attempts. The expected results demonstrate a significant reduction in data exposure risks without compromising system performance. It is concluded that adding an authentication layer to the clipboard buffer provides a robust defense against sensitive data being pasted into unauthorized AI chats or accessed by malicious background processes.

TABLE OF CONTENT

List of Figures vii

List of Tables vii

List of Abbreviations vii

List of Appendices vii

1. Introduction viii

    1.1 Background and Literature Survey

    1.2 Research Gap

    1.3 Research Problem

2. Objectives x

    2.1 Main Objectives

    2.2 Specific Objectives

3. Methodology xi

    3.1 Development Environment and Language Selection

    3.2 System Architecture and Workflow

    3.3 Security Escalation Protocol

    3.4 Data Escalation Protocol

    3.5 Data Retention and Logging

    3.6 Testing and Evaluating Strategy

4. Description and Personal and Facilities xiii

5. Budget and Budget Justification xiv

Reference List xv

Appendix A: User Interface Mockup and Wireframes xvi

    A.1 The Confirmation Pop-up

    A.2 Authentication Prompt

    A.3 Alert Notifications

Appendix B: Security Escalation Logic and xvii

B.1 Encryption Trigger (AES-256-GCM)
B.2 Failure Counter and Escalation Logic
B.3 API Communication (SMTP/Email)

LIST OF FIGURES

Figure 1.1: Logical Diagram of Clipboard Vulnerability.

Figure 1.2: System Architecture and Workflow

LIST OF TABLES

Table 3.3: Failed attempts and results

Table 4.1: Estimated Project Budget and Resource Costs.

LIST OF ABBREVIATIONS

AES: Advanced Encryption Standard

API: Application Programming Interface

DLP: Data Loss Prevention

GCM: Galois/Counter Mode

IDE: Integrated Development Environment

LLM: Large Language Model

RAM: Random Access Memory

SDK: Software Development Kit

SMTP: Simple Mail Transfer Protocol

LIST OF APPENDICES

Appendix A: User Interface Mockup and Wireframes

Appendix B: Security Escalation Logic and Pseudocode

INTRODUCTION

Background & Literature Survey

The system clipboard serves as a universal intermediary for data exchange between heterogeneous applications within an operating system. Despite its ubiquity, the standard clipboard architecture operates on a "Global Buffer" model, where data is typically stored in volatile memory as unencrypted plain text. In the current cybersecurity landscape, this lack of protection presents two primary risks: "Clipboard Hijacking" and "Intentional Data Leakage."

Existing literature identifies that malicious software (malware) can monitor system hooks to exfiltrate sensitive strings, such as cryptocurrency wallet addresses, login credentials, and private keys, directly from the clipboard buffer. Since data in a clipboard is freely delivered between arbitrary programs, any malicious application can easily watch or modify content during copy-paste operation **[1]**. Furthermore, with the proliferation of Large Language Models (LLMs) and AI chatbots, users frequently copy sensitive corporate data into external web interfaces, often bypassing traditional Data Loss Prevention (DLP) systems. Recent studies indicate that clipboard hijacking can be executed remotely via protocols like RDP without requiring local malware installation **[2]**. Current security measures, such as clearing the clipboard after a predefined time interval (e.g., 30 seconds), are reactive and fail to secure the data while it resides in the buffer. There is a clear need for a proactive mechanism that treats the clipboard as a secure "at-rest" storage location through immediate encryption.

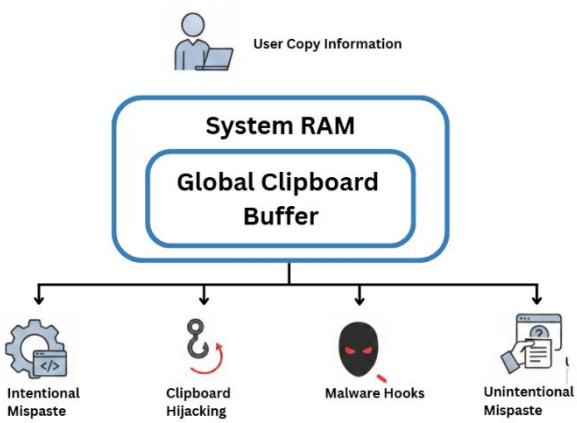

Figure 1.1: Logical Diagram of Clipboard Vulnerability

Research Gap

Despite the availability of various clipboard management tools, a significant void exists in the integration of proactive security and user-intent verification. Current solutions primarily fall into two categories, both of which fail to address the core problem of your project:

1. Productivity-Oriented Managers: These tools focus on maintaining a history of copied items, which increases the security risk by storing multiple sensitive plain-text strings in a local database.
2. Time-Based Clearers: Some security suites offer a feature to clear the clipboard after a set number of seconds; however, they do not protect the data during the period it remains active in the buffer, nor do they prevent the user from "intentionally" pasting sensitive data into unauthorized locations like public AI chats.

There is currently no lightweight, cross-platform software that implements Mandatory Encryption at the point of capture combined with a Confirmation-and-Password gate at the point of pasting. Furthermore, existing tools lack an automated security escalation path, such as triggering email alerts or account lockouts, specifically for clipboard-related authentication failures **[1]**. This "Clipboard Security Manager" fills this gap by treating the clipboard as a secure, vaulted environment rather than a transient plain-text storage area.

Research Problem

The fundamental architecture of the system clipboard presents a critical security paradox: it is designed for maximum accessibility while offering zero confidentiality. This creates three primary research problems that this project aims to address:

1. Plain-Text Vulnerability at Rest: Once data is copied, it resides in the system RAM as unencrypted plain text. Any application with basic user-level permission can programmatically "peek" at the clipboard buffer, leading to the silent theft of credentials or sensitive strings by background malware.
2. Lack of Intent Verification: Current operating systems assume that if a "Paste" command is issued, the user intends for the data to be transferred.

There is no native gateway to prevent "accidental" pastes of sensitive data into high-risk environments, such as public AI chat interfaces or unauthorized messaging platforms.

3. Absence of an Audit Trail and Response: Standard clipboard buffers do not log access attempts or provide a mechanism to alert the user of suspicious activity. If a malicious script attempts to read the clipboard repeatedly, the user remains unaware, and there is no automated defensive escalation, such as email notifications or temporary access lockouts.

OBJECTIVES

Main Objectives

The primary goal of this research is to design and develop a lightweight, cross-platform background application that secures the system clipboard by implementing immediate encryption upon data capture and requiring multi-stage user authentication before data is decrypted and pasted.

Specific Objectives

- To develop a high-performance background service using Go (Golang) that hooks into system-level clipboard events across Windows, Linux, and macOS.
- To implement AES-256-GCM encryption to secure the clipboard buffer "at rest," ensuring that any third-party application attempting to read the buffer receives only encrypted ciphertext.
- To design a "Challenge-Response" user interface that triggers a confirmation pop-up and password entry field during every paste attempt.
- To integrate an automated security response system that utilizes an API to send email alerts on the 2nd failed attempt and initiates a temporary account lockout or password reset on the 3rd failure.
- To establish a resource-efficient 30-day rolling log policy that tracks security events while maintaining a minimal hardware footprint

METHODOLOGY

The development of the "Clipboard Security Manager" follows a systematic software development life cycle (SDLC) focusing on system-level integration and cryptographic security.

Development Environment and Language Selection

The application is developed using **Go (Golang)** to ensure high performance and native cross-platform compatibility. Go's ability to handle system-level "hooks" efficiently makes it ideal for monitoring clipboard events without significant CPU overhead, ensuring the software remains a "small software which runs in background."

System Architecture and Workflow

The system operates on a "Capture-Encrypt-Verify-Paste" model.

1. Data Capture & Encryption**:** The application monitors the OS clipboard hook. When a copy event occurs, the data is instantly intercepted and encrypted using AES-256-GCM. The application utilizes the Advanced Encryption Standard (AES) with a 256-bit key length, which is the industry benchmark for symmetric cryptographic robustness **[3]**.This ensures that any other application attempting to read the clipboard buffer will only access encrypted ciphertext.
2. Paste Interception**:** When the user attempts to paste, the application intercepts the command and generates a graphical pop-up interface (to gain confirmation).
3. Authentication Layer**:** The user must enter a password. The system retrieves the Argon2 hash of the stored password via an API and compares it with the hash of the user's input to ensure security even if the local database is compromised.

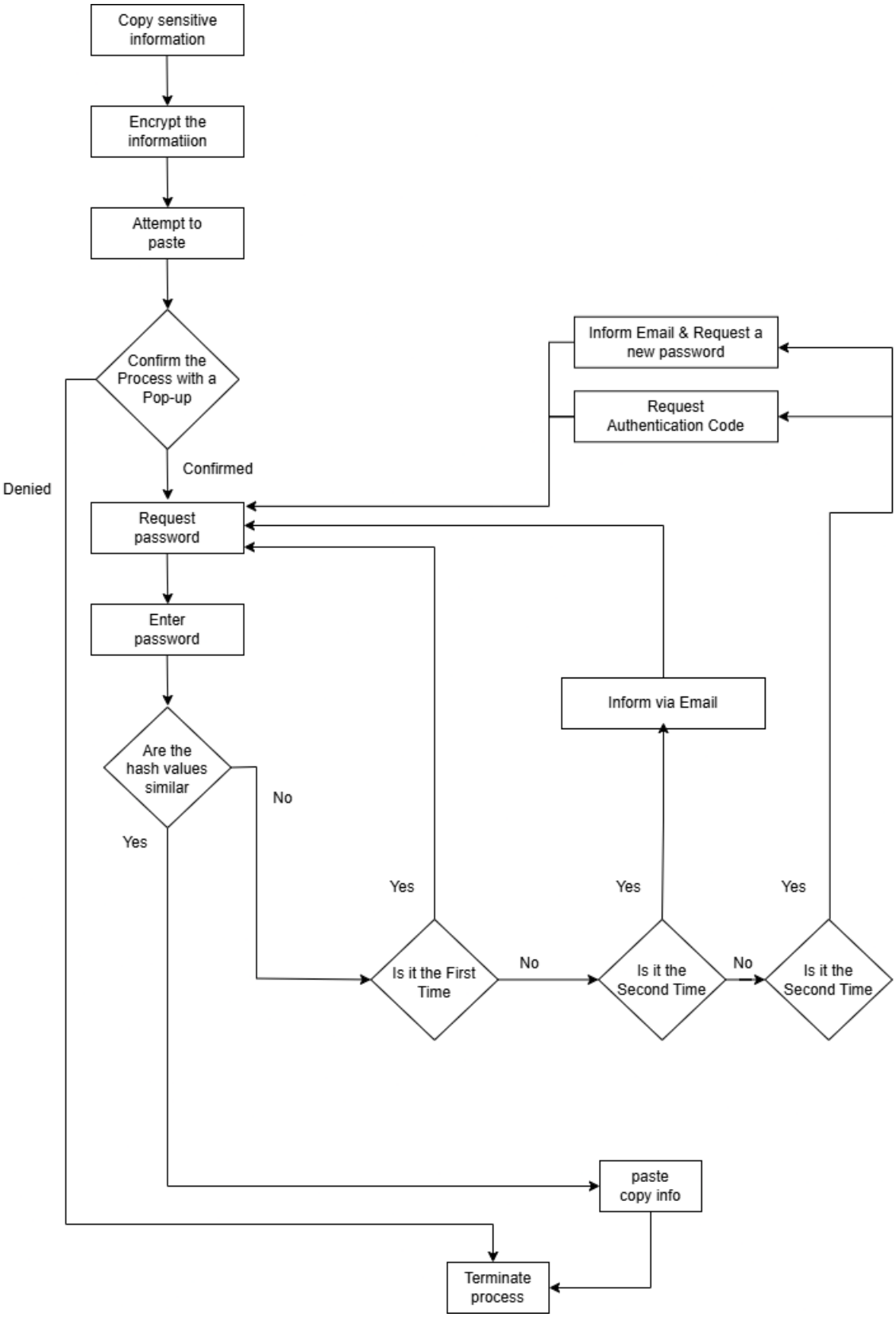

Figure 3.2 System Architecture and Workflow

Security Escalation Protocol

To prevent unauthorized access, the system implements a tiered response when entering the password:

Attempt	Result
1st Failed Attempt	A simple error message is displayed, and a second attempt is granted
2nd Failed Attempt	The system triggers an automated SMTP/API email alert to the user's registered email address notifying them of a suspicious access attempt.
3rd Failed Attempt	The application triggers a temporary account lockout and requires a password reset or a temporary authentication code to restore functionality.

Table 3.3: Failed attempts and results

Data Retention and Logging

To maintain a minimal hard disk footprint, the software maintains an encrypted rolling log. Security events, such as failed login attempts and paste confirmations, are stored for a maximum of 30 days before being automatically overwritten.

Testing and Evaluation Strategy

"Clipboard Security Manager" meets the required performance and security benchmarks, the following tests will be conducted:

1. Encryption Latency Test**:** Measuring the time delay between the "Copy" command and the completion of AES-256-GCM encryption Comparative analysis shows that AES provides a superior balance of encryption speed and

security compared to other symmetric algorithms like DES or 3DES **[4]**.

2. Buffer Leak Test**:** Attempting to read the clipboard using unauthorized third-party scripts to verify that only ciphertext is accessible.
3. Authentication Stress Test**:** Validating the tiered response system (email triggers and lockouts) by simulating repeated failed password entries.
4. Cross-Platform Compatibility Test**:** Verifying consistent behavior of system hooks across Windows, Linux (Kali), and macOS environments

DESCRIPTION OF PERSONAL AND FACILITIES

To successfully complete this project, the following resources and facilities will be utilized:

- Personnel**:** The project will be developed by two undergraduate researchers specializing in Cyber Security, responsible for full-stack development, cryptographic implementation, and testing.
- Hardware**:** A standard laptop with minimum 8GB RAM and an internet connection for API communication and literature research.
- Software Development Kit (SDK): The Go (Golang) programming environment and VS Code as the primary Integrated Development Environment (IDE).
- API Services: * SendGrid or Mailgun API: For triggering automated security email alerts.
    - External Hashing API: For secure, off-site password verification via Argon2.
- Testing Environment: A virtualized environment using Kali Linux and Windows 10/11 to verify cross-platform functionality and protection against clipboard hijacking scripts.

Budget and Budget Justification

As this project focuses on software development using open-source tools and existing personal hardware, the financial requirements are minimal. The project is designed to be completed within a "Zero-Budget" framework by leveraging free-tier developer services.

Item Category	Description	Estimated Cost	Justification
Development Tools	Go (Golang) SDK, VS Code IDE	0.00	Open-source software
Security APIs	SendGrid (Free Tire), Argon2API	0.00	Student/ Developer free tiers
Hardware	Personal Workstation (Laptop)	0.00	Pre-owned by researcher
OS Environment	Windows 11 & Kali Linux	0.00	Community/Standard licenses
Total		0.00	

Table 4.1: Estimated Project Budget and Resource Costs

The primary "cost" of the project is the researcher's time and effort. The choice of the Go language and AES-256 encryption ensures that the software remains lightweight, requiring no additional expensive high-performance hardware for execution or storage.

REFERENCE LIST

[1] R. S. S. Singh and S. S. Singh, "Implementation of Clipboard Security using Cryptographic Techniques," *International Journal of Computer Applications*, vol. 86, no. 6, pp. 24–28, Jan. 2014. [Online]. Available: IEEE Xplore.

[2] M. S. Ibrahim et al., "Clipboard Data Attacks and Detection via Remote Desktop Protocol," in *2023 5th International Conference on Computer Communication and the Internet (ICCCI)*, Fujisawa, Japan, 2023, pp. 112–117. doi: 10.1109/ICCCI59327.2023.10296672.

[3] J. Daemen and V. Rijmen, *The Design of Rijndael: AES - The Advanced Encryption Standard*. Berlin, Germany: Springer-Verlag, 2002.

[4] S. Mehrotra and R. Mishra, "Comparative Analysis of Encryption Algorithms For Data Communication," *International Journal of Computer Science and Technology (IJCST)*, vol. 2, no. 2, pp. 192–195, Jun. 2011.

APPENDIX A: USER INTERFACE MOCKUP AND WIREFRAMES

Since the application operates primarily in the background, the UI is triggered only during specific "Paste" events to ensure a "small software" footprint.

A.1. The Confirmation Pop-up

This window appears immediately after a user triggers a "Paste" command (Ctrl+V or Right-Click > Paste).

- Elements:
    - Header: "Security Alert: Sensitive Data Detected"
    - Message: "You are attempting to paste encrypted data. Do you wish to proceed?"
    - Buttons: [Confirm] [Deny]

A.2. Authentication Prompt

If the user selects "Confirm," the secondary authentication layer is presented.

- Elements:
    - Input Field: "Enter Security Password" (Masked characters)
    - Hint: "Verification required for decryption."
    - Button: [Decrypt & Paste]

A.3. Alert Notifications

- Desktop Alert: A small toast notification appearing in the system tray if access is denied.
    - Message: "Access Denied: Incorrect Password. Attempt 1 of 3."
- Email Notification (2nd Attempt): A mockup of the automated email sent via API.
    - Subject: "Security Alert: Unauthorized Paste Attempt Detected"

- Body: "An unverified paste attempt was made on your workstation at [Timestamp]. If this was not you, please secure your account."

Appendix B: Security Escalation Logic and Pseudocode

This section demonstrates the core implementation logic using Go (Golang) syntax as planned in your methodology.

B.1. Encryption Trigger (AES-256-GCM)

This logic executes the moment a "Copy" event is detected by the system hook.

Code:

```
// AES-256-GCM Encryption Trigger
func onCopyEvent(plainText []byte) []byte {
    key := getSecretKey() // 32-byte key for AES-256
    nonce := make([]byte, 12)

    block, _ := aes.NewCipher(key)
    aesgcm, _ := cipher.NewGCM(block)

    // Encrypt and return ciphertext to the clipboard buffer
    return aesgcm.Seal(nil, nonce, plainText, nil)
}
```

B.2. Failure Counter and Escalation Logic

This tracks the three variations of the password entry process you defined.

```
// Tiered Security Escalation Logic
func verifyPasteAttempt(userInput string) {
    storedHash := callHashingAPI(userID) // API Call for Argon2 Hash
```

```
    if compareHash(userInput, storedHash) {
        decryptAndPaste()
        resetFailCounter()
    } else {
        failCounter++
        handleFailure(failCounter)
    }
}

func handleFailure(count int) {
    switch count {
    case 1:
        showNotification("Incorrect Password. Try again.")
    case 2:
        showNotification("Incorrect Password. Security alert sent to email.")
        triggerEmailAPI(userEmail, "Suspicious Activity Detected") // API Call
    case 3:
        showNotification("Account Locked. Reset required.")
        lockClipboardSession()
        triggerEmailAPI(userEmail, "Account Lockout & Reset Code")
    }}
```

B.3. API Communication (SMTP/Email)

A simplified representation of how the background service interacts with the external API.

```
// API Call for Security Alerts
func triggerEmailAPI(email string, subject string) {
    payload := map[string]string{
        "to":      email,
        "subject": subject,
        "message": "Security protocol triggered due to failed authentication.",
    }
    // Execute POST request to SendGrid/Mailgun API
```

```
    http.Post("https://api.emailservice.com/send", "application/json", payload)
}
```